AMAN OS

Complete Project Information and Technical Dossier

Generated from the verified repository state

Repository: aman-search | Branch: gemini-free | Runtime: Node.js 22

Identity

- Project: **AMAN OS**, evolved from AMAN Search
- Repository: `https://github.com/smshk84-star/aman-search.git`
- Branch: `gemini-free`
- Local path: `C:\Users\smshk\Documents\Codex\2026-08-06\mujhe\outputs\aman-search-repo`
- Latest verified commit: `4e4d414 test: cover honest development mocks`
- Platform/runtime: Netlify + Node.js 22

Product

AMAN OS is a browser-local personal workspace combining a command center, source-backed web search, AI chat, conversations, projects, tasks, local files, activity history, deterministic agent routing, settings, and an installable PWA.

Runtime architecture

```
Browser UI (index.html/app.js/styles.css)
  -> /.netlify/functions/search or /.netlify/functions/ai
  -> Netlify validation, rate limiting, and SSE
  -> provider gateway
  -> Gemini adapter
  -> Gemini API + Google Search grounding
  -> streamed answer, citations, and source cards
```

The browser never receives the provider secret.

Main files

- `index.html` - application shell and metadata
- `app.js` - UI, navigation, search, chat, projects, tasks, files, agents
- `styles.css` - responsive visual system
- `manifest.webmanifest` - PWA metadata
- `sw.js` - service-worker cache and offline fallback
- `lib/os/store.mjs` - versioned browser-local persistence
- `lib/os/agents.mjs` - deterministic routing and agent runs
- `lib/os/files.mjs` - upload validation and filename safety
- `netlify/functions/search.mjs` - search API boundary
- `netlify/functions/ai.mjs` - AI chat API boundary
- `netlify/functions/lib/search-gateway.mjs` - search provider selection
- `netlify/functions/lib/ai-gateway.mjs` - AI provider selection

- `netlify/functions/lib/gemini-provider.mjs` - grounded search adapter
- `netlify/functions/lib/gemini-ai-provider.mjs` - streaming chat adapter
- `netlify/functions/lib/search-utils.mjs` - validation, safe URLs, SSE, retry, timeout, limits

AI/search status

The active committed implementation uses **Gemini with Google Search grounding**, not OpenAI Responses API.

- Search: Gemini grounded search
- Chat: Gemini streaming
- Secret: `GEMINI_API_KEY`
- Selectors: `SEARCH_PROVIDER`, `AI_PROVIDER`
- Optional model: `AI_MODEL`
- Optional fallbacks: `SEARCH_FALLBACK_PROVIDER`, `AI_FALLBACK_PROVIDER`

Without a key, local development uses a clearly labelled mock. Production returns a configuration error and never fabricates live results.

Untracked experimental files exist but are not active runtime code:

```
netlify/functions/lib/citation-mapper.mjs
netlify/functions/lib/llm-client.mjs
netlify/functions/lib/tavily-client.mjs
```

Data model and boundaries

Workspace data is stored in the current browser profile under `localStorage` key `aman-os:v1`. It contains projects, conversations, messages, tasks, files, preferences, activity, and agent runs.

This is a single-user MVP: no cloud database, login, multi-user authorization, or cross-device sync. Clearing site storage removes the workspace. Do not use it for sensitive data.

Files are browser-local, limited to 1.5 MB, name-sanitized, allowlisted by document/image type, and never executed or uploaded to the server.

Security

`netlify.toml` configures CSP, HSTS, `X-Content-Type-Options`, `X-Frame-Options`, strict referrer policy, restricted permissions, and cross-origin policies.

Functions enforce same-origin checks, request/payload limits, safe HTTP/HTTPS sources, escaped rendering, sanitized provider errors, timeouts, cancellation, and in-memory rate limits of 8 search requests/minute and 12 AI requests/minute per function instance.

PWA

The service worker uses cache `aman-os-v1`, caches the static shell, keeps function requests network-only, and provides offline navigation fallback. Bump the cache version when static assets materially change.

Netlify configuration

`netlify.toml` uses root publish directory, `netlify/functions`, Node 22, and esbuild. Add the following only in Netlify Project configuration -> Environment variables:

```
GEMINI_API_KEY=your Google AI Studio API key
```

Never put this value in frontend code, GitHub, URLs, browser storage, or `.env.example`.

Local development

Requirements: Node.js 22 and Netlify CLI.

```
npm.cmd run check
npm.cmd test
netlify dev
```

Use a non-committed `.env` with `GEMINI_API_KEY` for real local provider testing; without it the labelled mock is expected.

Verification

Verified commands:

```
npm.cmd run check
npm.cmd test
git diff --check
```

Current result: syntax checks passed, **25 tests passed**, 0 failed, 0 skipped, and diff check passed. Tests cover citations, safe URLs, SSE parsing, provider errors, retries, cancellation, rate limiting, search/AI contracts, mock honesty, persistence, file validation, tasks, and agent routing.

Git state and remaining work

Committed AMAN OS code is pushed to branch `gemini-free`. Existing untracked items were intentionally left untouched:

```
.kiro/
netlify/functions/lib/citation-mapper.mjs
netlify/functions/lib/llm-client.mjs
netlify/functions/lib/tavily-client.mjs
```

To activate live AI/search, obtain a Google AI Studio key, add it to Netlify as `GEMINI_API_KEY`, deploy `gemini-free`, and test search plus chat.

Documentation

- `README.md` - setup, provider, deployment, verification
- `docs/AMAN_SEARCH_BASELINE.md` - original baseline
- `docs/ARCHITECTURE.md` - boundaries and persistence
- `docs/PROVIDERS.md` - provider contracts
- `docs/SECURITY.md` - security model
- `CHANGELOG.md` - history